

Overview

With our team formed and enough practice with GenAI, in both a code focused and design-focused manner, we are ready to take on a project utilizing modern Agile methodologies.

While the project deliverable in the form of working quality software is quite important, students are strongly reminded that the process we undertake is the actual focus in our academic exploration of SWE. **A repeatable and observable quality focused process that ends up producing less feature rich software will get a higher grade than a large feature rich piece of software done opaquely or with less process.**

TL;DR - For a course, **process >> product**. Beyond a course, that may not hold.

Agile and Process Requirements

Your team will be required to

- * Perform a documented sprint planning meeting before starting work each sprint and capture the information in your repository (GitHub)
- * Hold stand-up meetings virtually (Slack) and/or in-person at least 3 times a week and capture the information in your repository (GitHub)
- * Tasks are captured in an issue tracker (GitHub Issues) and work happens and is documented in the issue tracker
- * Perform a sprint review and retrospective at least two times in the remaining time of the quarter. Information from the retrospective must be captured and there must be evidence of its incorporation
- * Meet with your TA weekly and capture the meeting information (GitHub)
- * Meet with the Prof once before the conclusion of the quarter

Some meetings may not include all team members. For example, TA or Prof meetings may be attended by two or three people, but planning and retrospective meetings must be attended by all.

Repository, Process, and Tooling Requirements

- All work including planning, meeting information, tests, and documentation must be captured in GitHub incrementally
- GenAI may be used, and if used, must be exposed and discussed

- Regardless of production mechanism (human or AI) work batches above 300 LoC must follow a pull-request path with evaluation by another human on the team
- Branching should be demonstrated and used over the project process but approach can vary as you go
- Versioning using [SemVer](#) must be employed
- A CI/CD pipeline must be built using GitHub Actions
- Deployment can happen to GitHub pages, Cloudflare, or as a downloadable asset, depending on the form.
- Testing with unit and e2e must be demonstrated, and not be applied only at the conclusion of the project. Significant early efforts with testing should be verifiable in the repository. Testing approaches may vary in tooling.
- Code documentation must be maintained as you go along in the project. This also includes commenting using JSDocs
- Linting and quality checks should be performed both manually by developers and via a CI pipeline for software artifacts (this may go beyond JavaScript code)
- Repositories should use a .gitignore file
- Commit messages should be consistent and follow a format like [Conventional Commits](#)
- A changelog should be kept and may be generated manually, automatically, or some hybrid
- A technical documentation site should be found either in a Github Wiki or private web site for future maintainers

Technical Constraints

- You may use markdown, standards-based HTML, CSS without a framework, vanilla JavaScript without a framework, media assets such as audio, video, image, fonts, or PDF files, JSON files, and any .txt or configuration files needed
- Any server-side based technologies must work on Cloudflare or Github pages only
- All major technical decisions must be captured as Architectural Decision Records (ADRs) in [MADR](#) format
- Dependencies can only be added with agreement from your teaching assistant (which implies proper justification)
- The form of what you build may include a traditional web application, progressive web application (Web app), wrapped mobile app (CapacitorJS), or a wrapped desktop app (ElectronJS). Chrome Extensions, VS Code extensions, and Slack extensions also have been seen and on occasion, REST endpoints for outside consumption make sense. Forms beyond any mentioned here must be given clearance.
- All projects must have a website to document what is produced for end users

Project Topic and Process

You will be given a topic as a prompt for you to clarify and design. Your TA mentor is your customer, and they work for the Prof who has driven the ideas. The topic itself will have many design decisions and open-ended aspects. You should do research and employ user-centered design throughout the project. This should include, minimally, a design brief, user personas, user stories, and wireframes. These artifacts should precede any non-exploratory work produced in the repository.

You will run a one-week sprint, Sunday to Sunday, for orienting, which is light work-wise due to the midterm. Then, a one-week design and prototyping sprint with initial exploration. Your TA will gate you on these two sprints and, if you do not perform them before heavy coding, you will be forced to repeat it the week after.

Your sprints will continue after, with a review break sometime in week 9. The review break will involve another team in your cohort that will use your software, look at your code, evaluate it, and provide product and code feedback. You will do the same. Interaction, or even software evaluation, should happen once from the Prof before the end of week 10, and you are expected to interact with the TA weekly.

Your Project Prompt

SE SitRep - What is the situation of your high-performing software team? What is everyone doing? Have we checked in today? Are there blockers? Are we feeling bad? Do we need someone to cover for us? When can we all meet? There are many challenges in keeping a small agile team on track; maybe software can help! The product formally known as Status Hero now called Steady - <https://runsteady.com/> shows that this is really a problem to be solved. How does it change with AI agents in the mix? Do they need tracking too?

Final Advice

Ok, hi teams! This is Jared now. Everything above is pretty much word-for-word from Powell.

I'm particularly excited about this project for a couple of reasons. First of all, just by studying the domain, I expect you'll all learn a *lot* about common industry practices, especially pertaining to Agile teams (which many of you will experience as you enter the workforce). My biggest priority in this course is that you all feel like you've *learned* about software engineering, and I think this project is one of the best topics to ensure that.

I understand though that many of you would prioritize resume development over learning objectives. That is **100% reasonable**. If I was in your shoes, I would feel *exactly* the same way. I personally think this project would look great on a resume as well, for a couple of reasons.

1. It is *not* a simple application. There are a lot of moving parts to consider and a lot of decisions to be made (about architecture, systems, features, etc.)

2. It demonstrates *knowledge* about software engineering processes, especially common issues.
3. Its a big team project (which always looks good)

That being said, this isn't me providing clearance to you all to use every popular technology possible. Start with vanilla web tech, and if you find a certain framework would significantly improve your project, we can talk about it.

For this first week, I just want you all to start thinking about the project. **Do not** start writing code yet - it is *critical* that you thoroughly plan first. There will be more information about the project as we progress. I would love it if you guys started planning your stand-up meetings / sorting out some logistics, and maybe start researching the domain this week. But if you feel overwhelmed (I know many of you have lots of midterms this week), just let it sit in the back of your mind and focus on those first.

If you guys have any questions - please please please come to me! Consider me a "stakeholder" in the product your team is creating. I'm really excited to see what you guys come up with!